III.6. Reprezentări în virgulă mobilă

Probleme cu reprezentările în virgulă fixă

- lungimea totală $n+m$ este fixată prin hardware
- dar, în virgulă fixă, atât n cât și m sunt la rîndul lor fixate
  - deci magnitudinea și precizia sunt prestabilite și nu pot fi modificate
  - dacă dorim o precizie mai bună și suntem dispuși să reducem magnitudinea (sau invers)?

Notaţia ştiinţifică

- utilă pentru numere cu multe cifre

  $745194.2623 \times 10^{-32}$ în loc de

  0.00000000000000000000000007451942623

- acelaşi număr - mai multe forme de scriere

  $571.42 \times 10^{2} = 5.7142 \times 10^{4} = 571420 \times 10^{-1} = \ldots$

- scriere normalizată - unică

  – exact o cifră semnificativă înainte de virgulă

  5.7142×10^{4}

Notaţia ştiinţifică în binar

- cifra semnificativă dinaintea virgulei poate fi doar 1
  - deci în practică nu este necesară memorarea sa
- excepție - reprezentarea numărului 0
  - doar cifre de 0
- scriere normalizată (număr nenul)

  $1.xx...x \times 2^y$
  - baza 2 este implicită - nu trebuie memorată

Reprezentări în virgulă mobilă

- componente
  - semnul (S): 0 sau 1 (+ sau -)
  - mantisa (M): 1.*xx*...*x*
    - de obicei se folosește partea fracționară (f)

    M = 1 + f; f = 0.*xx*...*x*
  - caracteristica
    - reprezentarea exponentului în exces

$$N = (-1)^S \times 1.f \times 2^{C\text{ - exces}}$$

Limite

- numărul de biți al caracteristicii este fixat
  - deci există o valoare minimă și una maximă pentru exponent
- depășire superioară - exponent prea mare
  - numărul este considerat $\pm\infty$
- depășire inferioară - exponent prea mic
  - numărul este considerat 0
- tipul depășirii nu depinde de semn

Standardizare

- esențială pentru portabilitate
- standardul IEEE 754/1985
  - elaborat între 1977 și 1985
  - prima implementare comercială: Intel 8087
- 2 variante principale
  - simplă precizie (32 biți)
  - dublă precizie (64 biți)
  - au fost proiectate și unele extensii

Simplă precizie

31	30 … 23	22 … 0
S	C = exponent + 127	f = partea fracţionară a mantisei

- corespunde tipului *float* din C/C++
- limite în baza 10
  - minim: $\approx 1.2 \times 10^{-38}$
    - orice număr mai mic în modul va fi considerat 0
  - maxim: $\approx 3.4 \times 10^{38}$
    - orice număr mai mare în modul va fi considerat $\pm\infty$

Dublă precizie

63	62 … 52	51 … 0
S	C = exponent + 1023	f = partea fracţionară a mantisei

- corespunde tipului *double* din C/C++
- limite în baza 10
  - minim: $\approx 1.7 \times 10^{-308}$
  - maxim: $\approx 1.7 \times 10^{308}$
- magnitudine mai mare
- precizie superioară

Structură

- de fapt, reprezentarea în virgulă mobilă este formată din două reprezentări în virgulă fixă
  - semnul și mantisa - reprezentare modul-semn
  - caracteristica - reprezentare în exces
- de ce câmpurile sunt în ordinea S,C,f?
  - pentru a compara două reprezentări, câmpurile trebuie luate în considerare în această ordine

Caracteristici IEEE 754/1985

	Simplă precizie	Dublă precizie
Biți semn+mantisă	24	53
Exponent maxim	128	1024
• numere finite	127	1023
Exponent minim	-127	-1023
• numere normalizate	-126	-1022
Exces caracteristică	127	1023

Exemplu 1

- fie numărul -23.25
  - cum se reprezintă în simplă precizie?
- semnul: 1 (negativ)
- scriere în baza 2: $-23.25_{(10)} = -10111.01_{(2)}$
- normalizare: $10111.01 = 1.011101 \times 2^4$
- caracteristica: $4 + 127 = 131 = 10000011_{(2)}$
- reprezentarea

  $1\ 10000011\ 0111010...0_{(2)} = C1BA0000_{(16)}$

Exemplu 2

- ce număr corespunde reprezentării $42D80000_{(16)}$ (simplă precizie)?

$42D80000_{(16)} = 0\ 10000101\ 10110000...0_{(2)}$

$S = 0 \rightarrow$ număr pozitiv

$C = 10000101_{(2)} = 133_{(10)} \Rightarrow e = 133 - 127 = 6$

$M = 1 + 0.1011 = 1.1011$

- numărul: $+1.1011 \times 2^6 = 1101100_{(2)} = 108_{(10)}$

Aritmetica extinsă

- în plus față de aritmetica numerelor reale
  - reprezentarea numărului ∞ și definirea regulilor elementare de calcul cu acesta
    - x / ∞, $x \times \infty$, $\infty \pm \infty$
  - reprezentare pentru rezultatul operaţiilor nedefinite (NaN - Not a Number) şi definirea regulilor de propagare a acestuia
    - NaN *op* x = NaN, $\forall op$
- utilizare - bibliotecile de funcţii matematice

Exemplu

- calculul funcției arccos cu formula

$$\arccos(x)=2\cdot\arctan\sqrt{(1-x)/(1+x)}$$

- care este valoarea arccos(-1)?

$$x=-1 \Rightarrow (1-x)/(1+x) = 2/0=\infty \Rightarrow$$

$$\Rightarrow \arctan\sqrt{(1-x)/(1+x)} = \pi/2$$

- răspuns: arccos (-1) = π
  - nu ar fi fost posibil de obținut fără aritmetica extinsă

Tipuri de valori în virgulă mobilă

tip valoare	exponent (e)	f	valoare
normalizată	$e_{min} < e < e_{max}$	orice valoare	$(-1)^S \times 1.f \times 2^e$
denormalizată	$e = e_{min}$	$f \neq 0$	$(-1)^S \times 0.f \times 2^e$
zero	$e = e_{min}$	$f = 0$	S 0 (= 0)
infinit	$e = e_{max}$	$f = 0$	S ∞ ($\pm\infty$)
NaN	$e = e_{max}$	$f \neq 0$	NaN

Depășiri

- depăşire inferioară
  - în forma normalizată, exponentul negativ nu poate fi reprezentat în câmpul caracteristicii
    - numărul va fi considerat 0
- depăşire superioară
  - în forma normalizată, exponentul pozitiv nu poate fi reprezentat în câmpul caracteristicii
    - numărul va fi considerat $\pm\infty$, după caz

Reprezentări denormalizate

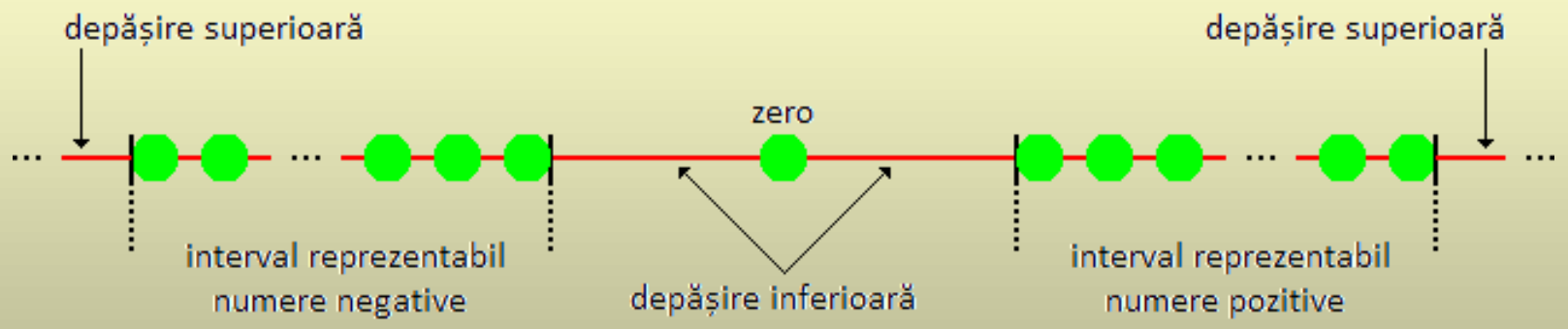
- număr mai mic în modul decât cea mai mică reprezentare normalizată nenulă
  - se renunţă la normalizare
  - exponentul are valoarea minimă
    - simplă precizie: -127
    - dublă precizie: -1023
  - în acest caz, mantisa va fi 0.f, în loc de 1.f

Aproximări (1)

- depășirea inferioară este de fapt aproximare
  - un număr nenul foarte mic este considerat 0
- care este precizia reprezentării în virgulă mobilă?
  - depinde de exponent
  - simplă precizie: 2^{e-23}
  - dublă precizie: 2^{e-52}
- exponent foarte mare - precizie foarte slabă

Aproximări (2)

- exemplu: e = 123
  - diferența între două numere consecutive reprezentabile exact este $2^{123-23} = 2^{100} \approx 10^{30}$
- ce putem reprezenta exact?
  - numere raționale (nu reale) - doar o parte



Aritmetica în virgulă mobilă

- fie două numere

$x=m_x \cdot 2^{e_x}$

$y=m_y \cdot 2^{e_y}$

- operațiile aritmetice elementare

$x+y=(m_x \cdot 2^{e_x-e_y}+m_y) \cdot 2^{e_y}$, dacă $e_x \leq e_y$

$x-y=(m_x \cdot 2^{e_x-e_y}-m_y) \cdot 2^{e_y}$, dacă $e_x \leq e_y$

$x \cdot y=(m_x \cdot m_y) \cdot 2^{e_x+e_y}$

$x:y=(m_x:m_y) \cdot 2^{e_x-e_y}$

Adunarea în virgulă mobilă

- compararea exponenților
  - se egalizează prin deplasarea unei mantise
- adunarea mantiselor
  - în complement față de 2
- normalizarea sumei
  - dacă apare depășire - oprire
- rotunjirea mantisei rezultat la numărul permis de biți

Înmulţirea în virgulă mobilă

- adunarea exponenților
- înmulțirea mantiselor
- normalizarea produsului
  - dacă apare depășire - oprire
- rotunjirea mantisei rezultat la numărul permis de biți
- determinarea semnului rezultatului

Temă

- urmăriți pașii adunării și respectiv înmulțirii în virgulă mobilă pentru reprezentările numerelor scrise în baza zece ca -0,75 şi 0,375
- se va considera reprezentarea în simplă precizie

IV. Arhitectura și organizarea calculatorului

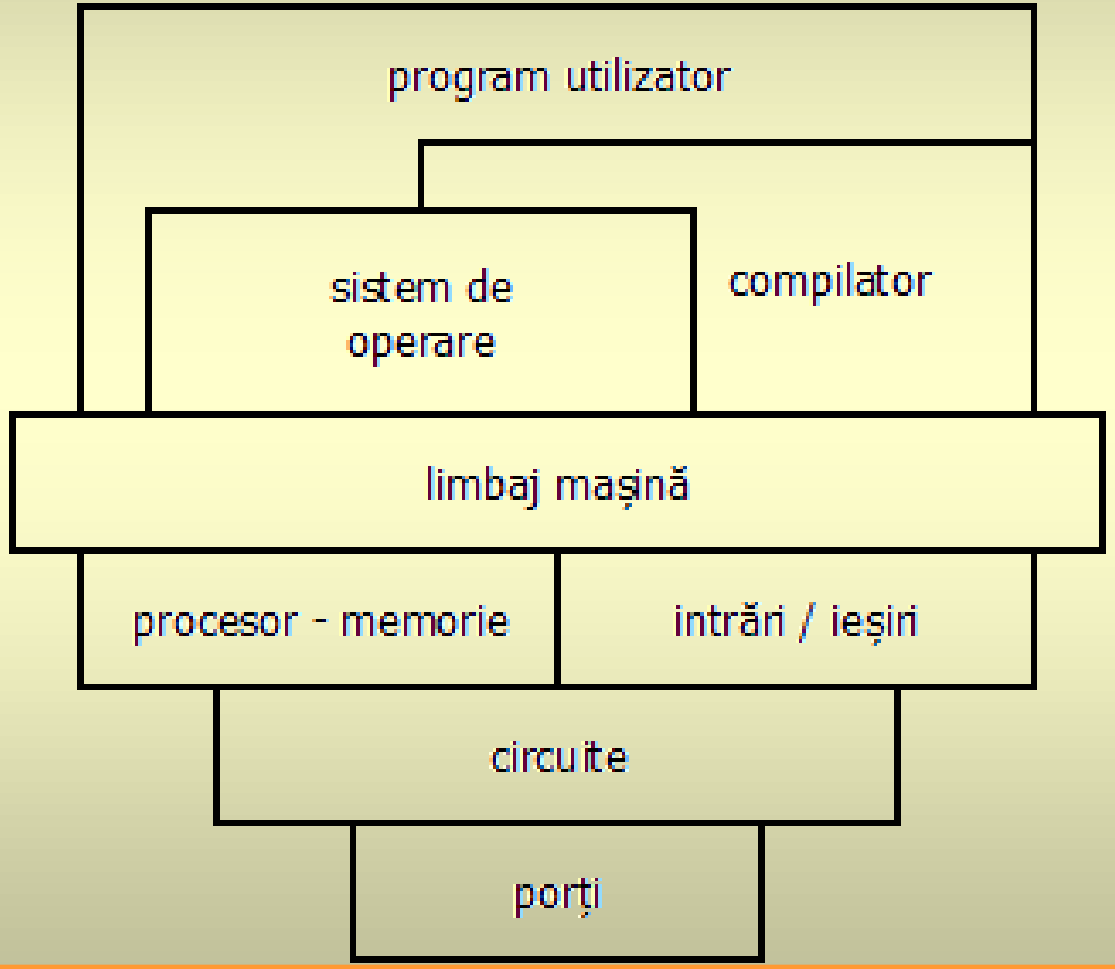
Modelul von Neumann (1)

- program memorat
  - codurile instrucțiunilor sunt reținute în aceeași memorie ca și datele
- memoria
  - ideal infinită; timp de acces egal la orice locație
- după execuția unei instrucțiuni urmează
  - instrucțiunea memorată imediat după ea
  - sau instrucțiunea indicată de cea curentă (salt)

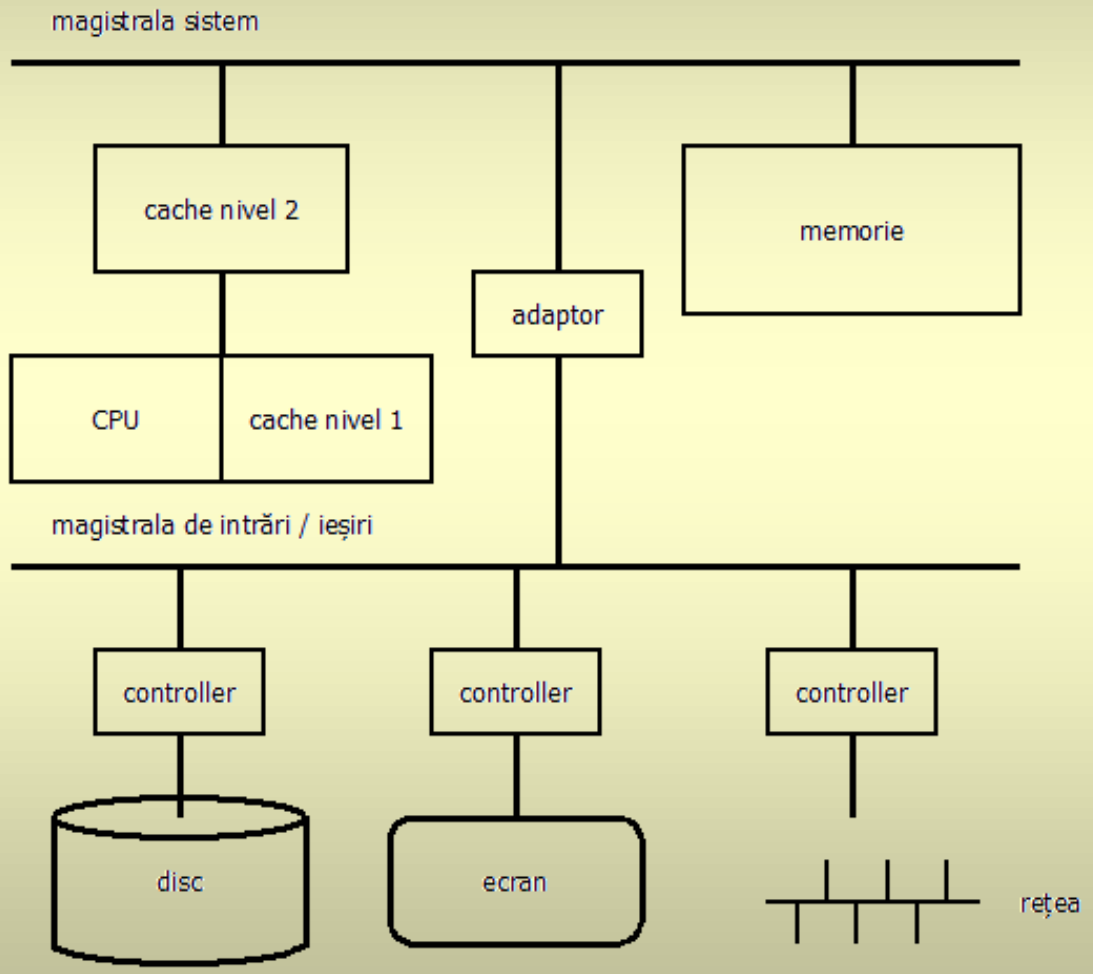
Modelul von Neumann (2)

- adresa instrucţiunii următoare este reținută în permanență
  - într-un registru dedicat - PC (Program Counter)
  - și actualizată permanent, în funcție de tipul instrucțiunii curente ("normală" sau de salt)
- în fiecare moment, o singură instrucţiune este încărcată pentru execuţie

Arhitectura unui sistem de calcul



Organizarea unui calculator



Arhitectura sistemului - componente

- arhitectura setului de instrucțiuni (ISA)
  - materializată prin diverse limbaje
  - în ordine crescătoare a nivelului de abstractizare
    - limbajul mașină
    - limbajul de asamblare
    - limbaje de nivel înalt
- organizarea mașinii

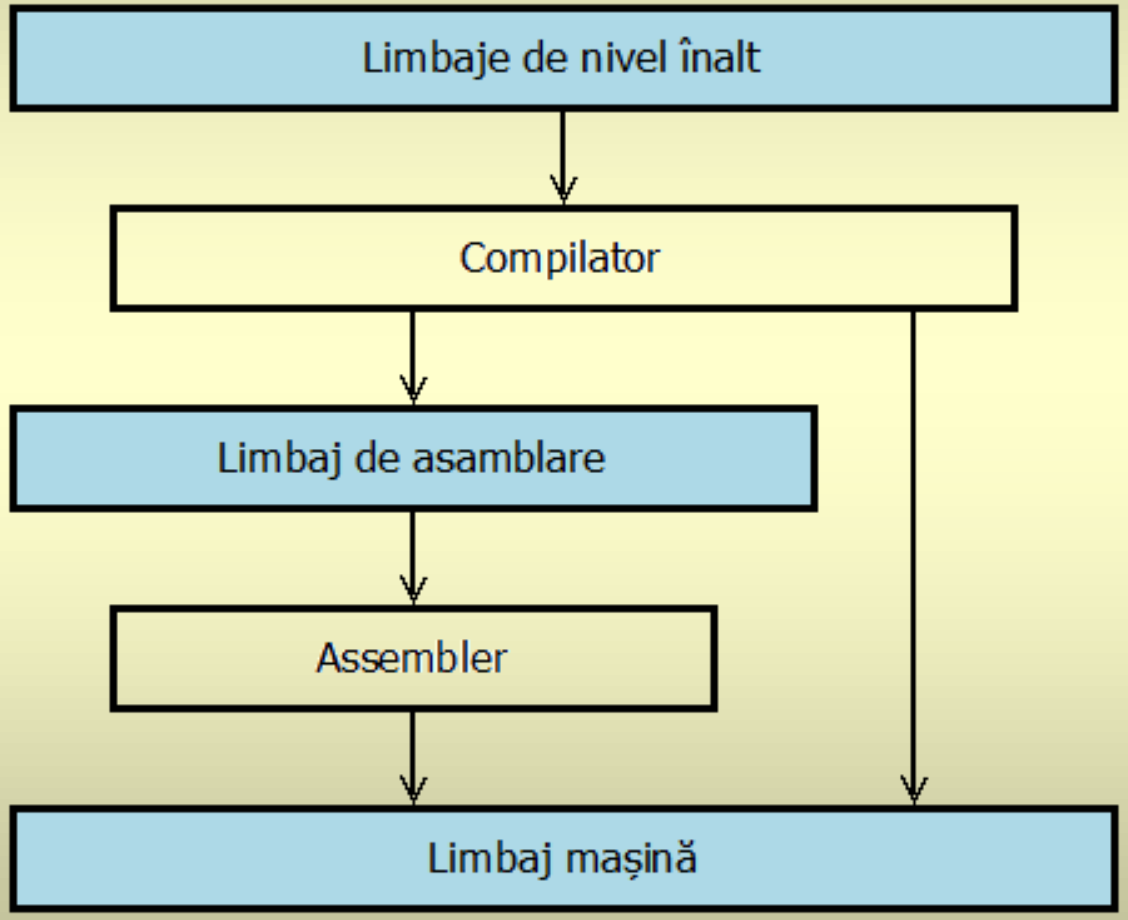
Limbaje de nivel scăzut

- limbajul mașină
  - caracteristic fiecărui procesor
  - constă din cuvinte peste alfabetul {0, 1}
- limbajul de asamblare
  - nivel (puțin) mai înalt
  - corespondență cu limbajul mașină (instrucțiuni)
- exemple
  - limbaj mașină: **0100 1010 1101 0010**
  - limbaj de asamblare: **sub myvar,5**

Traducere (1)

- singurul limbaj înțeles de procesor este limbajul mașină
  - pentru a fi executat, un program trebuie tradus
- asamblor (*assembler*)
  - traduce limbajul de asamblare în limbaj mașină
- compilator (*compiler*)
  - traduce un limbaj de nivel înalt în limbaj mașină
  - direct sau via limbajul de asamblare

Traducere (2)


Limbaje de nivel înalt
Compilator
Limbaj de asamblare
Assembler
Limbaj maşină

Limbaje de nivel înalt - avantaje

- dezvoltare mai rapidă a programelor
  - instrucțiuni mai ușor de înțeles și mai puține
- întreținere mai ușoară a programelor
  - aceleași motive
- portabilitatea programelor
  - puţine detalii dependente de maşină
    - dar nu deloc - portabilitatea nu e chiar 100%
  - fiecare limbaj necesită un compilator specific

Limbaj de asamblare - avantaje

- eficiență
  - cod mai compact - ocupă mai puțină memorie
  - viteză - execuție mai rapidă
  - și compilatoarele urmăresc eficiența
    - dar programatorul poate obține rezultate mai bune
- acces la resursele hardware
  - exemplu: procesoarele au biți care indică transportul și depășirea la adunare
    - nu sunt accesibile din limbajele de nivel înalt

Optimizare

- dorim ca sistemul să fie cât mai performant
  - dar nu putem îmbunătăți toate componentele
- care componente merită efortul de a fi îmbunătățite?
  - cele care au o pondere mai mare în funcționarea sistemului
  - cum măsurăm această pondere?

Legea lui Amdahl (1)

- considerăm un sistem (hardware, software) şi o anumită componentă a sa
- componenta respectivă lucrează un procentaj f_a din timpul de lucru al sistemului
- şi este îmbunătăţită, astfel încât lucrează de *a* ori mai rapid decât înainte
- de câte ori mai rapid devine sistemul?

Legea lui Amdahl (2)

$$A(a, f_a) = \frac{1}{(1 - f_a) + \frac{f_a}{a}}$$

- creştere de viteză generală cât mai mare
  - îmbunătăţirea pronunţată a componentei (a)
  - îmbunătăţirea componentelor cu o pondere (f_a) cât mai mare
    - deci mai des folosite